

Projet Flutter – Investment Companion

Module Compilation et Développement d'Applications Mobiles

1. Contexte et objectif

L'investissement peut sembler complexe lorsque l'on débute : actions, ETF, cryptomonnaies, diversification, risque, rendement, volatilité, durée de placement, inflation, frais, taux de change... Votre mission consiste à imaginer, concevoir et développer une application mobile Flutter destinée à aider un utilisateur novice à faire ses premiers pas dans cet univers.

Le concept fonctionnel est libre. L'application pourra notamment permettre de découvrir des actifs, analyser leur évolution, comparer des investissements, construire un portefeuille fictif, réaliser des simulations, expliquer des notions financières ou proposer des scores, classements et recommandations.

L'application ne permettra aucune transaction financière réelle et ne nécessitera aucune connexion à un compte bancaire ou à un courtier.

L'objectif n'est pas de reproduire une plateforme de trading. Il s'agit de construire **une application utile, pédagogique, fonctionnelle et techniquement bien conçue.**

Investment Companion — le compagnon du débutant

Concevez un compagnon interactif qui aide un utilisateur à découvrir progressivement l'investissement. L'application peut expliquer des notions à partir de données réelles, proposer de petites simulations, permettre de suivre quelques actifs et contextualiser les informations affichées. Par exemple : « Cette action a gagné 18 % sur un an, mais qu'est-ce que cela signifie ? », « Que ce serait-il passé si j'avais investi 50 € par mois ? », ou « Pourquoi diversifier un portefeuille ? ».

L'objectif est de faire apprendre par l'interaction avec les données, plutôt que de proposer un simple cours.

2. Proposition du concept

Avant de commencer le développement, chaque groupe remettra une proposition d'une page maximum présentant :

- le problème traité et l'utilisateur cible ;
- les principales fonctionnalités envisagées ;
- les données externes nécessaires et les API envisagées ;
- les données qui seront conservées localement ;
- les principaux états de l'application ;
- les calculs métier envisagés ;

- une première idée des animations et visualisations envisagées.

La proposition devra être validée avant le début du développement.

Exemples de concepts (donnés à titre indicatif) : Portfolio Starter, Investment Explorer, DCA Simulator, Risk Explorer, Investment Coach, ETF Explorer, Crypto Starter, Investment Scoring, Investment Goals, Currency Risk ou Macro Dashboard. Cette liste n'est ni obligatoire ni exhaustive.

Concept	Exemple d'application
Portfolio Starter	L'utilisateur constitue un portefeuille fictif d'actions, ETF ou cryptomonnaies. Il saisit ses achats et visualise la valeur actuelle du portefeuille, ses gains ou pertes et sa répartition.
Investment Explorer	Une application permettant à un débutant d'explorer différents investissements. Pour chaque actif : présentation simple, cours, historique graphique, performances et quelques indicateurs faciles à comprendre. Des favoris permettent de constituer une liste à suivre.
Investment Companion	Un compagnon pédagogique qui accompagne progressivement le débutant : « Qu'est-ce qu'un ETF ? », « Que signifie diversifier ? », « Pourquoi mon investissement a-t-il baissé ? ». Des données réelles et de petites simulations permettent d'illustrer chaque notion plutôt que de proposer uniquement du contenu théorique.
DCA Simulator	L'utilisateur choisit un actif, une somme et une périodicité : « Et si j'avais investi 100 € chaque mois depuis janvier 2022 ? ». L'application utilise les cours historiques pour reconstruire les achats et visualiser le capital investi et sa valeur actuelle.
Risk Explorer	L'application aide à comprendre qu'un rendement potentiel doit être considéré avec le risque. Elle compare plusieurs actifs à partir de leur historique : variations, périodes de baisse, stabilité du cours, etc., avec des explications adaptées à un novice.
ETF Explorer	Une application centrée sur les ETF. L'utilisateur découvre ce qu'ils représentent, compare plusieurs expositions (S&P 500, monde, marchés émergents, obligations...) et peut construire une sélection ou un portefeuille fictif.
Crypto Starter	Un outil de découverte des cryptomonnaies : prix, évolution, capitalisation, favoris et portefeuille fictif. L'application peut particulièrement mettre en évidence les fortes variations de prix et aider à les interpréter.
Investment Scoring	L'application attribue aux actifs un score calculé à partir de critères définis par le groupe : performance historique, régularité, importance des baisses, diversification, etc. L'utilisateur doit pouvoir comprendre comment et pourquoi le score a été obtenu.
Investment Goals	L'utilisateur définit un objectif — par exemple disposer de 20 000 € dans 8 ans — et l'application simule différents montants de départ, versements réguliers et hypothèses de rendement pour montrer différents scénarios.
Currency Risk	L'application montre à un investisseur européen qu'un investissement coté en dollars dépend à la fois de l'évolution de l'actif et du taux EUR/USD. Elle compare la performance dans la devise d'origine et celle réellement observée en euros.

Concept	Exemple d'application
Macro Dashboard	Une application pédagogique reliant quelques indicateurs économiques — inflation, taux, croissance... — aux marchés financiers. L'utilisateur construit son tableau de bord et conserve localement les dernières données récupérées.

3. Catalogue initial d'instruments

Un catalogue initial de 30 instruments (actions, ETFs ou cryptos) peut être considéré afin de faciliter le démarrage. Il constitue un jeu de données de départ et non une limite : les groupes peuvent intégrer davantage d'actifs ou proposer un mécanisme de recherche.

4. API et données historiques

L'application devra consommer au minimum une API REST externe. Les données temps réel ne sont pas forcément nécessaires : des données quotidiennes ou hebdomadaires peuvent suffire, en fonction du concept d'application retenu, pour la plupart des analyses, simulations et graphiques.

Pour les actions et ETF, un historique peut par exemple contenir la date, le cours d'ouverture, le plus haut, le plus bas, le cours de clôture et le volume (OHLCV). Pour les cryptomonnaies, un historique de prix et de capitalisation peut être exploité.

5. Persistance locale et cache

La persistance constitue un objectif important du projet. L'application devra distinguer les données propres à l'utilisateur des données récupérées depuis Internet.

Exemples de données utilisateur

- favoris ;
- portefeuille fictif et transactions ;
- simulations sauvegardées ;
- objectifs ;
- préférences ;
- configuration éventuelle d'un scoring.

Données provenant des API

Les données récupérées depuis Internet devront être mises en cache lorsque cela est pertinent. C'est notamment le cas des historiques de cours. L'application ne devra pas télécharger plusieurs fois les mêmes historiques sans raison.

La stratégie de cache pourra tenir compte de la date de la dernière donnée disponible et, lorsque l'API le permet, ne récupérer que les données manquantes. L'objectif est de limiter les appels réseau et les volumes transférés tout en conservant des informations suffisamment récentes, mais aussi de permettre un fonctionnement dégradé en cas d'absence de connexion.

La stratégie de persistance, d'invalidation et d'actualisation du cache devra être expliquée et justifiée dans le rapport.

6. Fonctionnement hors connexion

L'application devra conserver un intérêt lorsque le réseau est indisponible : favoris, portefeuille, simulations, historiques déjà téléchargés ou dernières valeurs connues pourront rester consultables.

L'utilisateur devra pouvoir identifier qu'il consulte éventuellement une donnée ancienne, par exemple en affichant la date et l'heure de dernière actualisation.

Lors de la soutenance, le fonctionnement hors connexion pourra être testé après fermeture complète de l'application et désactivation du réseau.

7. Gestion des états

La gestion des états constitue un objectif technique majeur. L'application devra représenter correctement les états associés aux traitements asynchrones : état initial, chargement, données disponibles, erreur et actualisation.

Elle devra également gérer ses états métier : portefeuille vide ou renseigné, actif favori ou non, simulation configurée ou non, résultat disponible ou non, etc.

Les groupes sont libres d'utiliser StatefulWidget / setState, Provider, Riverpod, Bloc/Cubit ou une autre solution. La sophistication n'est pas un objectif en soi : le choix sera évalué sur sa pertinence, la qualité de son implémentation et la capacité du groupe à l'expliquer.

8. Calculs métier

L'application devra réaliser au moins deux calculs métier significatifs afin de guider/conseiller les utilisateurs. Il ne vous est pas demandé d'avoir des connaissances préalables en finance : les exemples ci-dessous peuvent être repris ou adaptés. Il ne s'agit que d'exemples et vous êtes libres d'imaginer des indicateurs à ce niveau.

Calcul	Formule / principe	Exemple d'usage
Performance d'un investissement	$\text{performance (\%)} = (\text{valeur finale} - \text{valeur initiale}) / \text{valeur initiale} \times 100$	100 € deviennent 120 € : performance = 20 %.
Plus-value / moins-value	$\text{gain} = \text{valeur actuelle} - \text{montant investi}$	1 000 € investis, valeur actuelle 1 180 € : gain = 180 €.
Valeur d'un portefeuille	$\text{valeur} = \sum (\text{quantité possédée} \times \text{cours actuel})$	Additionner la valeur courante de chaque ligne du portefeuille.
Répartition d'un portefeuille	$\text{poids (\%)} = \text{valeur de l'actif} / \text{valeur du portefeuille} \times 100$	Permet de construire un graphique de répartition.
Rendement composé	$\text{capital final} = \text{capital initial} \times (1 + \text{taux})^{\text{nombre d'années}}$	1 000 € à 5 % pendant 10 ans ≈ 1 629 €.
Variation entre deux périodes	$\text{variation (\%)} = (\text{cours actuel} - \text{cours précédent}) / \text{cours précédent} \times 100$	Permet d'afficher la hausse ou baisse entre deux dates.

Simulation DCA	quantité achetée = montant périodique / cours à la date d'achat	Répéter le calcul à chaque période puis additionner les quantités.
----------------	---	--

9. Scoring et recommandations

L'application peut proposer des scores, classements ou suggestions d'investissement. Dans ce cas, le score doit être explicable : l'utilisateur doit pouvoir comprendre les critères utilisés et la raison du résultat.

Les critères, formules, normalisations et pondérations devront être documentés. Par exemple, un score pourrait combiner performance historique, stabilité, diversification et un autre critère défini par le groupe.

Un score basé sur des données historiques constitue un modèle pédagogique : il ne doit pas être présenté comme une prédiction certaine des performances futures.

10. Animations

L'application devra intégrer au minimum trois animations pertinentes, dont au moins deux devront être programmées par vous.

Par « programmées », on entend des animations dont le comportement est explicitement contrôlé, par exemple avec `AnimationController`, `Tween`, `AnimatedBuilder`, `AnimatedWidget` ou un mécanisme Flutter équivalent.

- apparition ou disparition progressive d'une fiche ;
- transition entre deux valeurs ou deux états ;
- évolution animée d'un indicateur ;
- déploiement d'informations supplémentaires ;
- animation lors de l'ajout aux favoris ;
- animation d'un résultat de simulation.

Une animation fournie entièrement par un package externe peut compter parmi les trois animations, mais pas parmi les deux animations codées. Les animations doivent avoir un intérêt pour l'expérience utilisateur.

11. Contraintes imposées pour le projet

- au moins 3 écrans significatifs et une navigation entre les écrans ;
- un formulaire avec validation ;
- une liste dynamique ;
- au moins une API REST et conversion JSON vers des objets Dart ;
- persistance de données métier ;
- cache local de données issues d'une API ;
- fonctionnement partiel hors connexion ;
- gestion explicite des états de chargement, de données et d'erreur ;
- au moins une visualisation graphique significative ;
- au moins deux calculs métier ;
- au moins trois animations, dont deux codées ;
- des tests unitaires portant sur la logique métier.

12. Architecture et qualité logicielle

L'architecture de l'application constitue un critère d'évaluation à part entière. Une séparation cohérente est attendue entre interface utilisateur, gestion des états, logique métier, accès aux données distantes et persistance locale.

Aucune architecture particulière n'est imposée. Les groupes devront cependant être capables de justifier de leurs choix. L'ensemble de la logique ne devra pas être concentré dans quelques Widgets.

Les choix pourront notamment être discutés à partir des questions suivantes :

- où se trouve l'état de l'application et pourquoi ?
- quelle partie décide d'interroger l'API ou d'utiliser le cache ?
- où se trouvent les calculs métier ?
- quelle partie est responsable de la persistance ?
- comment les erreurs et états de chargement sont-ils propagés vers l'interface ?
- que se passe-t-il lorsqu'un Widget est reconstruit ?

13. Tests

Des tests unitaires devront porter sur la logique métier. Les fonctions de calcul constituent de bons candidats : performance, valeur de portefeuille, DCA, variation, conversion de devise, drawdown ou score.

Pensez également aux cas limites : historique vide, montant nul, valeur manquante, dates incohérentes, division par zéro, etc.

14. Rapport

En plus du code source, chaque groupe remettra un rapport présentant le travail réalisé et justifiant les principaux choix de conception. Le rapport est un livrable évalué.

Il devra notamment présenter :

- le concept, l'utilisateur cible et les objectifs ;
- les fonctionnalités prévues et celles effectivement réalisées ;
- des captures des principaux écrans ;
- l'architecture logicielle retenue et un schéma des principaux composants/classes ;
- la stratégie de gestion des états et sa justification ;
- le choix de la solution de persistance et le modèle des données persistées ;
- la stratégie de cache, d'invalidation et d'actualisation ;
- les API utilisées et les données exploitées ;
- les principaux traitements et calculs métier ;
- les animations réalisées et leur implémentation ;
- les éventuels algorithmes de scoring ou recommandation ;
- les tests réalisés ;
- les difficultés rencontrées, limitations et améliorations possibles.

Le rapport ne devra pas seulement décrire ce qui a été fait : il devra expliquer et justifier les choix techniques.

15. Démonstration

La soutenance comportera une démonstration fonctionnelle suivie d'une discussion technique. Les questions pourront porter sur l'architecture, la gestion des états, le cycle de vie des Widgets, les appels asynchrones, la persistance, le cache, le fonctionnement hors connexion, le modèle de données, les animations et les tests.

Certaines modifications simples du code ou certains scénarios pourront être demandés afin de vérifier la compréhension de l'implémentation.

Annexe A — Quelques notions financières utiles

Cette annexe fournit un vocabulaire minimal. Ces notions ne constituent pas des prérequis disciplinaires : elles servent de support pour imaginer et programmer des fonctionnalités.

A.1 Volatilité

La volatilité cherche à répondre à la question : « le prix de cet investissement varie-t-il beaucoup ? ». Un actif dont les rendements changent fortement d'une période à l'autre est plus volatil. Une approche classique consiste à calculer l'écart-type des rendements périodiques. Pour ce projet, une définition simplifiée et correctement expliquée est acceptable.

A.2 Maximum drawdown

Le maximum drawdown mesure la plus forte baisse observée entre un sommet et le point bas qui suit. Exemple : 100 → 110 → 125 → 115 → 90 → 105. Le sommet est 125 et le point bas suivant 90 ; la baisse est $(90 - 125) / 125 = -28\%$. Le maximum drawdown est donc ici de -28 %.

A.3 DCA — investissement périodique

Le DCA (Dollar-Cost Averaging) consiste à investir régulièrement une même somme. Pour chaque date, on calcule la quantité achetée : montant investi / cours à cette date. Les quantités sont ensuite additionnées et valorisées au cours final.

A.4 Diversification

La diversification consiste à répartir un investissement entre plusieurs actifs, secteurs, zones géographiques ou classes d'actifs. Dans un projet étudiant, elle peut être approchée simplement à partir du nombre de positions et de leur poids dans le portefeuille.

A.5 Attention aux données brutes et ajustées

Les cours historiques peuvent être bruts ou ajustés pour tenir compte d'événements tels que les divisions d'actions et les dividendes. Pour un exercice pédagogique, l'utilisation de cours bruts est possible, mais le rapport doit indiquer les limites de cette approche lorsqu'elle affecte les calculs de performance.

Annexe B — API et sources de données suggérées

Les offres gratuites et limitations des fournisseurs évoluent. Les informations ci-dessous ont été vérifiées en septembre 2026 ; elles devront être revérifiées avant le démarrage du projet.

Source	Données	Intérêt / limites pour le projet	Documentation
Alpha Vantage	Actions / ETF / séries temporelles	Clé API requise. TIME_SERIES_DAILY : OHLCV quotidien ; mode compact disponible avec les 100 derniers points. La documentation couvre également recherche de symboles et séries hebdomadaires/mensuelles.	Documentation officielle
Twelve Data	Actions / ETF / forex / crypto	Plan Basic gratuit annoncé à 8 crédits API/minute et 800/jour. Utile comme alternative pour les séries temporelles et la recherche de symboles.	Documentation officielle
Financial Modeling Prep (FMP)	Actions / historiques EOD / données de référence	Plan Basic gratuit annoncé à 250 appels/jour et données historiques de fin de journée. Des endpoints historiques EOD simplifiés sont documentés.	Documentation officielle
CoinGecko	Cryptomonnaies	Compte Demo et clé API pour le plan gratuit actuel. Endpoints de marché et historiques, notamment market_chart et market_chart/range.	Documentation officielle
Frankfurter	Devises / taux de change	API publique sans clé. Taux actuels et historiques, séries temporelles, données provenant de banques centrales et sources officielles.	Documentation officielle
World Bank Indicators API	Indicateurs macroéconomiques	Pas de clé API. Accès à près de 16 000 séries d'indicateurs ; de nombreuses séries remontent à plus de 50 ans.	Documentation officielle

